```
# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like
competitions, datasets, and models to your session
# Learn more about kagglehub:
https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/jsphyg/weather-dataset-rattle-package/
weatherAUS.csv
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Dataset load karein (Aapki file ka path)
df = pd.read_csv("/kaggle/input/datasets/jsphyg/weather-dataset-
rattle-package/weatherAUS.csv")

# Data ka ek chota hissa dekhne ke liye
print("Dataset ke pehle 5 rows:")
print(df.head())
```

```
Dataset ke pehle 5 rows:
         Date Location  MinTemp  MaxTemp  Rainfall  Evaporation
Sunshine  \
0  2008-12-01   Albury     13.4     22.9       0.6          NaN
NaN
1  2008-12-02   Albury      7.4     25.1       0.0          NaN
NaN
2  2008-12-03   Albury     12.9     25.7       0.0          NaN
NaN
3  2008-12-04   Albury      9.2     28.0       0.0          NaN
NaN
4  2008-12-05   Albury     17.5     32.3       1.0          NaN
NaN

  WindGustDir  WindGustSpeed WindDir9am  ... Humidity9am  Humidity3pm
\
0           W           44.0          W  ...        71.0         22.0

1         WNW           44.0        NNW  ...        44.0         25.0

2         WSW           46.0          W  ...        38.0         30.0

3          NE           24.0         SE  ...        45.0         16.0

4           W           41.0        ENE  ...        82.0         33.0


   Pressure9am  Pressure3pm  Cloud9am  Cloud3pm  Temp9am  Temp3pm
RainToday  \
0       1007.7       1007.1       8.0       NaN     16.9     21.8
No
1       1010.6       1007.8       NaN       NaN     17.2     24.3
No
2       1007.6       1008.7       NaN       2.0     21.0     23.2
No
3       1017.6       1012.8       NaN       NaN     18.1     26.5
No
4       1010.8       1006.0       7.0       8.0     17.8     29.7
No

   RainTomorrow
0            No
1            No
2            No
3            No
4            No

[5 rows x 23 columns]
```

```python
# 1. Jo rows kaam ki nahi hain ya jisme RainTomorrow hi missing hai, unhe hata dete hain
df = df.dropna(subset=['RainTomorrow'])

# 2. Sirf simple numerical (numbers wale) columns chun rahe hain taaki seekhna aasan ho
features = ['MinTemp', 'MaxTemp', 'Rainfall', 'WindGustSpeed', 'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm', 'Pressure9am', 'Pressure3pm']
X = df[features]
y = df['RainTomorrow']

# 3. Jo bachi hui missing values hain, unhe har column ke average (mean) se bhar dete hain
X = X.fillna(X.mean())

# 4. Target variable ('RainTomorrow') ko Numbers me badalte hain: Yes -> 1, No -> 0
y = y.map({'Yes': 1, 'No': 0})

print("Data clean ho gaya hai!")
print(f"Total rows training ke liye: {len(X)}")
```

```
Data clean ho gaya hai!
Total rows training ke liye: 142193
```

```python
# Data split kar rahe hain
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training data size: {X_train.shape}")
print(f"Testing data size: {X_test.shape}")
```

```
Training data size: (113754, 10)
Testing data size: (28439, 10)
```

```python
# Model ka object banayein
model = LogisticRegression(max_iter=1000)

# Machine ko sikhayein (Training)
print("Machine train ho rahi hai...")
model.fit(X_train, y_train)
print("Machine train ho chuki hai! □")
```

```
Machine train ho rahi hai...
Machine train ho chuki hai! □
```

```python
# Test data par predict karein
y_pred = model.predict(X_test)

# Accuracy check karein
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"\nMachine ki Accuracy: {accuracy * 100:.2f}%")

# Detailed report (Ki kitne Yes aur kitne No sahi predict huye)
print("\nDetailed Report:")
print(classification_report(y_test, y_pred, target_names=['No Rain
(0)', 'Rain (1)']))
```

```
Machine ki Accuracy: 83.51%

Detailed Report:
              precision    recall  f1-score   support

 No Rain (0)       0.86      0.94      0.90     22098
    Rain (1)       0.70      0.46      0.55      6341

    accuracy                           0.84     28439
   macro avg       0.78      0.70      0.73     28439
weighted avg       0.82      0.84      0.82     28439
```

```
# Ek naya sample data banate hain (Features ke sequence me: MinTemp,
MaxTemp, Rainfall, WindGustSpeed, WindSpeed9am, WindSpeed3pm,
Humidity9am, Humidity3pm, Pressure9am, Pressure3pm)
# Maan lijiye aaj Humidity3pm bohot zyada hai (85%) aur baarish bhi
thodi hui hai
naya_mausam = [[15.2, 24.8, 0.0, 44.0, 15.0, 20.0, 80.0, 85.0, 1022.0,
1007.0]]

# Predict karein
prediction = model.predict(naya_mausam)

if prediction[0] == 1:
    print("\nPrediction: Kal Baarish Hogi!  (Yes)")
else:
    print("\nPrediction: Kal Baarish Nahi Hogi! ☀ (No)")
```

```
Prediction: Kal Baarish Hogi!  (Yes)

/usr/local/lib/python3.12/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LogisticRegression was fitted with feature names
  warnings.warn(
```